


# Actividad 1. Primera PKI: creación y despliegue básico de certificados X.509

Ana Trinidad Alonso Ansón

Grado en Ciberseguridad, Universidad Internacional de La Rioja (UNIR)

Seguridad en Comunicaciones Inalámbricas

Prof. Álvaro Vallega Moreno

8 de noviembre de 2026

# Índice

Índice . . . . . 2

Actividad 1. Primera PKI: creación y despliegue básico de certificados X.509 . . . . . 3

Introducción . . . . . 3

1. Creación de la autoridad certificadora (CA) y emisión de certificados . . . . . 3

1.1. Estructura de directorios y archivo openssl.cnf . . . . . 3

1.2. Clave y certificado raíz . . . . . 5

1.3. Certificado del servidor . . . . . 6

1.4. Certificado de cliente (EAP-TLS) . . . . . 8

1.5. Verificación de la cadena y registro de la CA . . . . . 10

1.6. ¿Qué contiene un certificado X.509 y qué función tiene la CA? . . . . . 10

2. Configuración de un servidor HTTPS local . . . . . 11

2.1. Servidor con Python (http.server) . . . . . 11

2.2. Comprobación de la conexión TLS . . . . . 11

2.3. Importación de la CA en el navegador . . . . . 12

2.4. Acceso a https://localhost:8443 sin advertencias . . . . . 13

Conclusiones . . . . . 13

Referencias . . . . . 14

# Actividad 1. Primera PKI: creación y despliegue básico de certificados X.509

## Introducción

Este informe documenta la construcción de una pequeña infraestructura de clave pública (PKI) con OpenSSL: una autoridad certificadora (CA) raíz, un certificado de servidor para *localhost* y un certificado de cliente apto para autenticación EAP-TLS. Después se despliega un servidor HTTPS local con Python y se importa la CA en Chromium para comprobar que la conexión es de confianza. Todos los comandos y salidas se han ejecutado realmente; las capturas son del terminal y del navegador.

**Entorno.** Linux x86-64 con OpenSSL 3.0.13, Python 3 y Chromium (Playwright) con la base NSS del usuario (*~/.pki/nssdb*). Los comandos son idénticos en Kali Linux. Las contraseñas (*unir1234*) son solo de laboratorio.

## 1. Creación de la autoridad certificadora (CA) y emisión de certificados

### 1.1. Estructura de directorios y archivo openssl.cnf

Se crea el árbol de trabajo clásico de una CA: *certs/* (certificados), *private/* (claves, con permisos 700), *newcerts/* (copias de lo emitido), *csr/* (peticiones) y *crl/*. Además se inicializan *index.txt* (base de datos de emisiones) y *serial* (siguiente número de serie).

**Figura 1**

*Creación del entorno de trabajo de la CA*

```
terminal: ~/pki
$ mkdir -p certs private newcerts csr crl

$ chmod 700 private

$ touch index.txt && echo 1000 > serial && echo 1000 > crlnumber

$ ls -l
total 28
drwxr-xr-x 2 root root 4096 Nov  8 01:07 certs
drwxr-xr-x 2 root root 4096 Nov  8 01:07 crl
-rw-r--r-- 1 root root    5 Nov  8 01:07 crlnumber
drwxr-xr-x 2 root root 4096 Nov  8 01:07 csr
-rw-r--r-- 1 root root    0 Nov  8 01:07 index.txt
drwxr-xr-x 2 root root 4096 Nov  8 01:07 newcerts
drwx------ 2 root root 4096 Nov  8 01:07 private
-rw-r--r-- 1 root root    5 Nov  8 01:07 serial
```

*Nota.* Comandos mkdir, chmod, touch y ls -l ejecutados en ~/pki.

El archivo *openssl.cnf* se adapta a este entorno: define la sección *[CA_default]* con las rutas anteriores, SHA-256 y política laxa, y tres perfiles de extensiones: *v3_ca* (CA:true, keyCertSign, cRLSign), *server_cert* (serverAuth y *subjectAltName* con localhost, 127.0.0.1 y ::1, imprescindible porque los navegadores modernos ignoran el CN) y *usr_cert* (clientAuth, necesario en EAP-TLS).

**Figura 2**

*Archivo openssl.cnf (parte 1)*

terminal: ~/pki (openssl.cnf, parte 1)

```
$ cat openssl.cnf
[ ca ]
default_ca = CA_default
[ CA_default ]
dir               = .
certs             = $dir/certs
new_certs_dir     = $dir/newcerts
database          = $dir/index.txt
serial            = $dir/serial
private_key       = $dir/private/ca.key.pem
certificate       = $dir/certs/ca.cert.pem
default_md        = sha256
default_days      = 375
policy            = policy_loose
copy_extensions   = none
unique_subject    = no
[ policy_loose ]
countryName             = optional
stateOrProvinceName     = optional
organizationName        = optional
commonName              = supplied
[ req ]
default_bits        = 2048
distinguished_name  = req_dn
string_mask         = utf8only
default_md          = sha256
prompt              = no
[ req_dn ]
C  = ES
O  = UNIR Practicas PKI
CN = Placeholder
[ v3_ca ]
subjectKeyIdentifier   = hash
```

*Nota.* Configuración de la CA y políticas.

**Figura 3**

*Archivo openssl.cnf (parte 2)*

terminal: ~/pki (openssl.cnf, parte 2)

```
authorityKeyIdentifier = keyid:always,issuer
basicConstraints       = critical, CA:true
keyUsage               = critical, digitalSignature, cRLSign, keyCertSign
[ server_cert ]
basicConstraints       = CA:FALSE
subjectKeyIdentifier   = hash
authorityKeyIdentifier = keyid,issuer:always
keyUsage               = critical, digitalSignature, keyEncipherment
extendedKeyUsage       = serverAuth
subjectAltName         = @alt_names
[ alt_names ]
DNS.1 = localhost
IP.1  = 127.0.0.1
IP.2  = ::1
[ usr_cert ]
basicConstraints       = CA:FALSE
subjectKeyIdentifier   = hash
authorityKeyIdentifier = keyid,issuer
keyUsage               = critical, digitalSignature, keyEncipherment
extendedKeyUsage       = clientAuth, emailProtection
```

*Nota.* Perfiles de extensiones para CA, servidor y cliente.

### 1.2. Clave y certificado raíz

Se genera una clave RSA de 4096 bits protegida con AES-256 y un certificado autofirmado válido 10 años con *openssl req -x509*. La clave privada queda en modo 400 para que solo pueda leerla su propietario.

**Figura 4**

*Generación de la clave privada y del certificado raíz*

terminal: ~/pki

```
$ openssl genrsa -aes256 -passout pass:unir1234 -out private/ca.key.pem 4096

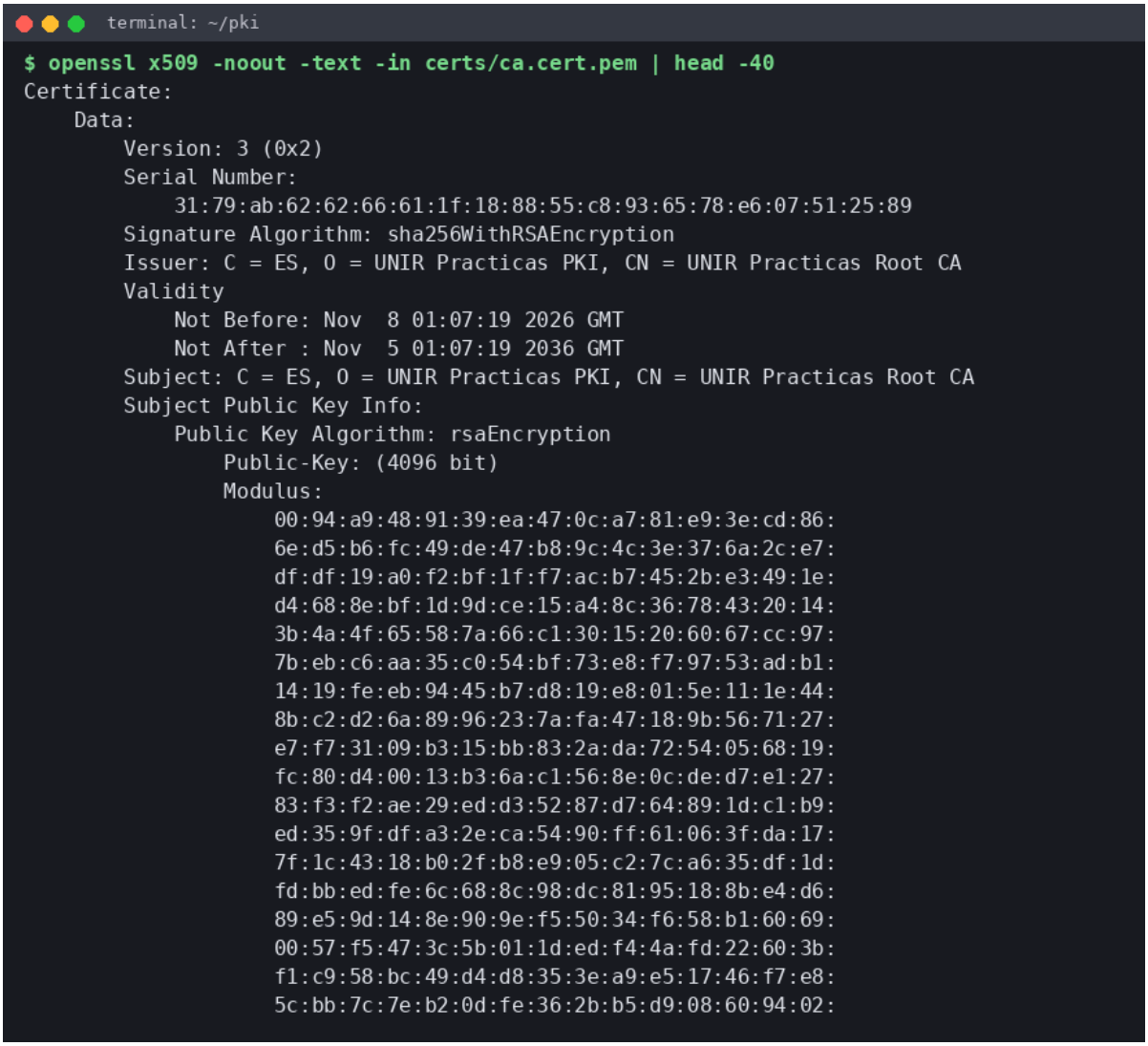
$ chmod 400 private/ca.key.pem

$ openssl req -config openssl.cnf -key private/ca.key.pem -passin pass:unir1234 -new
    -x509 -days 3650 -sha256 -extensions v3_ca -subj '/C=ES/O=UNIR Practicas PKI/CN=UNIR
    Practicas Root CA' -out certs/ca.cert.pem
```

*Nota.* Salida vacía significa éxito en openssl; el contenido se inspecciona a continuación.

**Figura 5**

*Contenido del certificado raíz (openssl x509 -text)*



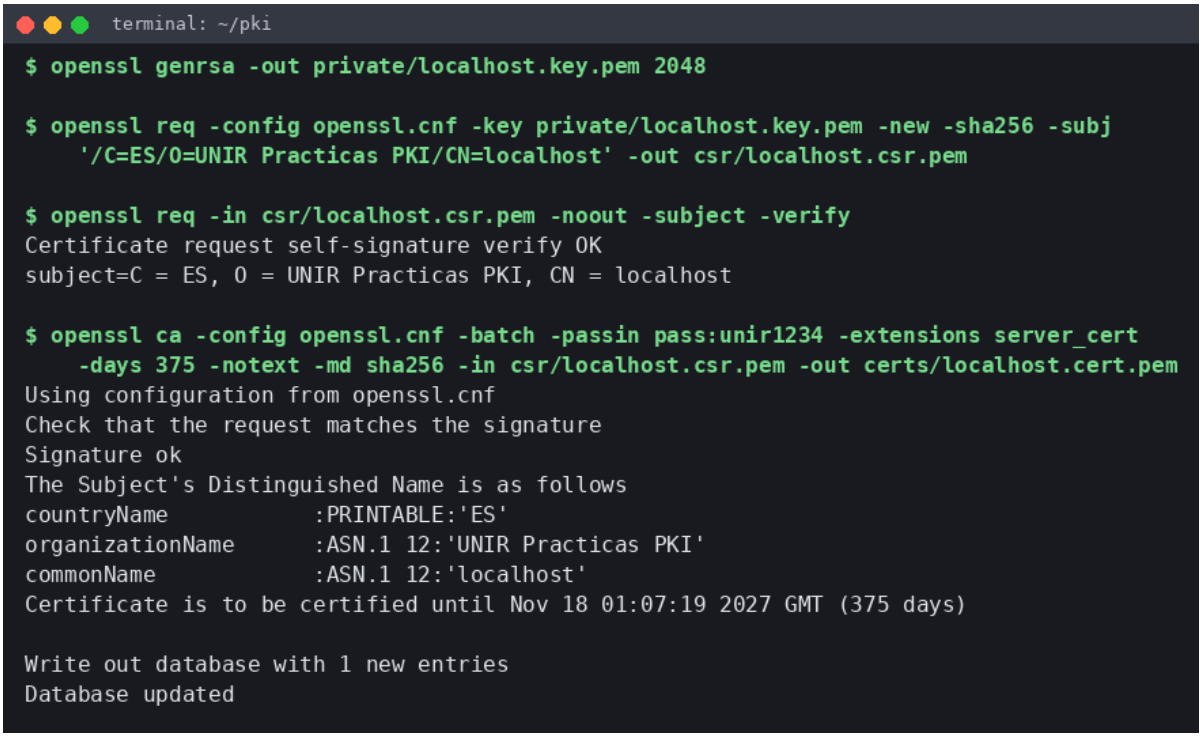

*Nota.* Se observan Issuer = Subject (autofirmado), CA:TRUE, Key Usage crítico y los identificadores SKI/AKI.

### 1.3. Certificado del servidor

Se crea una clave RSA de 2048 bits, una petición CSR con CN=localhost y se firma con *openssl ca* usando el perfil *server_cert*. La CA registra la emisión (número de serie 1000) en *index.txt*.

**Figura 6**

*Clave, CSR y firma del certificado del servidor*




*Nota.* La verificación de la CSR (verify OK) precede a la firma por la CA.

**Figura 7**

*Detalle del certificado del servidor*

```
terminal: ~/pki
$ openssl x509 -noout -text -in certs/localhost.cert.pem | sed -n '1,12p;/X509v3
    extensions/,/Signature Algorithm/p'
Certificate:
    Data:
        Version: 3 (0x2)
        Serial Number: 4096 (0x1000)
        Signature Algorithm: sha256WithRSAEncryption
        Issuer: C = ES, O = UNIR Practicas PKI, CN = UNIR Practicas Root CA
        Validity
            Not Before: Nov  8 01:07:19 2026 GMT
            Not After : Nov 18 01:07:19 2027 GMT
        Subject: C = ES, O = UNIR Practicas PKI, CN = localhost
        Subject Public Key Info:
            Public Key Algorithm: rsaEncryption
        X509v3 extensions:
            X509v3 Basic Constraints:
                CA:FALSE
            X509v3 Subject Key Identifier:
                45:CE:3B:50:CE:87:06:F3:D0:1C:18:B5:DB:19:A1:6A:C0:18:28:5B
            X509v3 Authority Key Identifier:
                keyid:47:BD:AA:F9:5B:10:DA:15:1B:58:34:7A:F4:5A:F3:0D:64:73:C4:6E
                DirName:/C=ES/O=UNIR Practicas PKI/CN=UNIR Practicas Root CA
                serial:31:79:AB:62:62:66:61:1F:18:88:55:C8:93:65:78:E6:07:51:25:89
            X509v3 Key Usage: critical
                Digital Signature, Key Encipherment
            X509v3 Extended Key Usage:
                TLS Web Server Authentication
            X509v3 Subject Alternative Name:
                DNS:localhost, IP Address:127.0.0.1, IP Address:0:0:0:0:0:0:0:1
    Signature Algorithm: sha256WithRSAEncryption
```

*Nota.* Extensiones: CA:FALSE, serverAuth y SAN DNS:localhost, IP:127.0.0.1, IP:::1.

### 1.4. Certificado de cliente (EAP-TLS)

Para la autenticación EAP-TLS se emite un certificado de cliente con el perfil *usr_cert* (extendedKeyUsage clientAuth). Se empaqueta junto con su clave y la CA en un archivo PKCS#12 (*cliente.p12*), que es el formato que importan navegadores, sistemas operativos y supplicants como wpa_supplicant.

**Figura 8**

*Emisión del certificado de cliente*

```
terminal: ~/pki
$ openssl genrsa -out private/cliente.key.pem 2048

$ openssl req -config openssl.cnf -key private/cliente.key.pem -new -sha256 -subj
    '/C=ES/O=UNIR Practicas PKI/CN=Ana Trinidad Alonso Anson' -out csr/cliente.csr.pem

$ openssl ca -config openssl.cnf -batch -passin pass:unir1234 -extensions usr_cert -days
    375 -notext -md sha256 -in csr/cliente.csr.pem -out certs/cliente.cert.pem
Using configuration from openssl.cnf
Check that the request matches the signature
Signature ok
The Subject's Distinguished Name is as follows
countryName           :PRINTABLE:'ES'
organizationName      :ASN.1 12:'UNIR Practicas PKI'
commonName            :ASN.1 12:'Ana Trinidad Alonso Anson'
Certificate is to be certified until Nov 18 01:07:20 2027 GMT (375 days)

Write out database with 1 new entries
Database updated

$ openssl x509 -noout -subject -issuer -dates -ext extendedKeyUsage -in
    certs/cliente.cert.pem
subject=C = ES, O = UNIR Practicas PKI, CN = Ana Trinidad Alonso Anson
issuer=C = ES, O = UNIR Practicas PKI, CN = UNIR Practicas Root CA
notBefore=Nov  8 01:07:20 2026 GMT
notAfter=Nov 18 01:07:20 2027 GMT
X509v3 Extended Key Usage:
    TLS Web Client Authentication, E-mail Protection
```

*Nota.* CN=Ana Trinidad Alonso Anson, emitido por UNIR Practicas Root CA (serie 1001).

**Figura 9**

*Empaquetado PKCS#12 del cliente*

```
terminal: ~/pki
$ openssl pkcs12 -export -inkey private/cliente.key.pem -in certs/cliente.cert.pem
    -certfile certs/ca.cert.pem -name 'Cliente EAP-TLS' -passout pass:unir1234 -out
    certs/cliente.p12

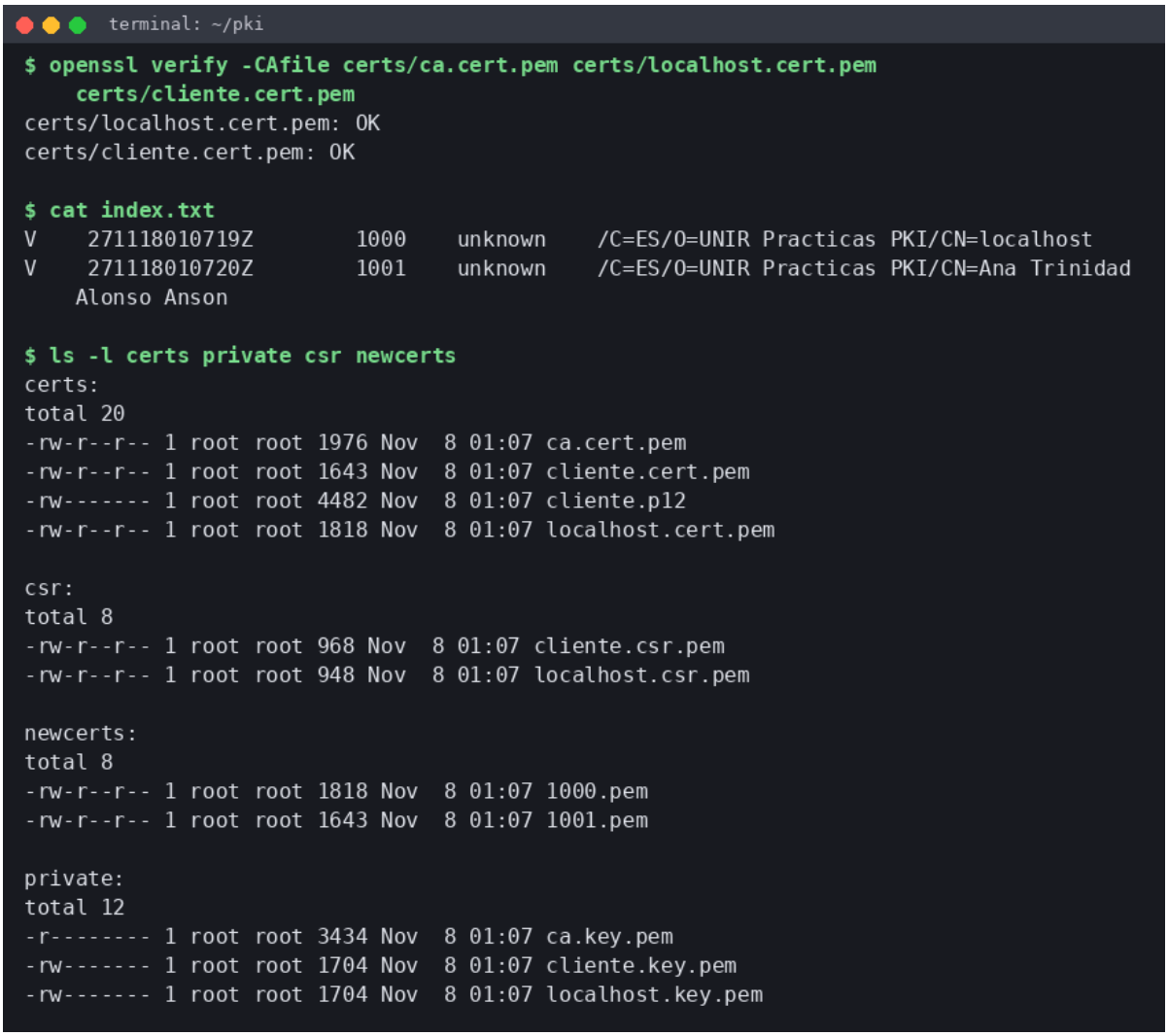
$ openssl pkcs12 -info -in certs/cliente.p12 -passin pass:unir1234 -nokeys -nodes | grep
    -E 'subject|issuer|MAC|Bag'
MAC: sha256, Iteration 2048
MAC length: 32, salt length: 8
PKCS7 Encrypted data: PBES2, PBKDF2, AES-256-CBC, Iteration 2048, PRF hmacWithSHA256
Certificate bag
Certificate bag
PKCS7 Data
Shrouded Keybag: PBES2, PBKDF2, AES-256-CBC, Iteration 2048, PRF hmacWithSHA256
Bag Attributes
subject=C = ES, O = UNIR Practicas PKI, CN = Ana Trinidad Alonso Anson
issuer=C = ES, O = UNIR Practicas PKI, CN = UNIR Practicas Root CA
Bag Attributes: <No Attributes>
subject=C = ES, O = UNIR Practicas PKI, CN = UNIR Practicas Root CA
issuer=C = ES, O = UNIR Practicas PKI, CN = UNIR Practicas Root CA
```

*Nota.* El .p12 contiene el certificado de cliente, la CA y la clave privada cifrada (shrouded keybag).

## 1.5. Verificación de la cadena y registro de la CA

**Figura 10**

*Verificación con openssl verify y estado de la CA*




*Nota.* Ambos certificados validan contra la CA; index.txt muestra las dos emisiones con estado V (válido).

## 1.6. ¿Qué contiene un certificado X.509 y qué función tiene la CA?

Un certificado X.509 (RFC 5280) liga una identidad a una clave pública mediante una firma digital. Incluye: versión (v3), número de serie, algoritmo de firma, emisor (*Issuer*), periodo de validez, sujeto (*Subject*), clave pública del sujeto y extensiones (*Basic Constraints*, *Key Usage*, *Extended Key Usage*, *Subject Alternative Name*, SKI/AKI), seguido de la firma del emisor (Housley et al., 2008).

La CA es el tercero de confianza: verifica la identidad del solicitante y firma su certificado con su clave privada. Quien confía en la CA (porque tiene su certificado raíz

instalado) puede validar cualquier certificado firmado por ella sin conocer previamente al titular. Por ello la clave privada de la CA se protege con contraseña y permisos estrictos. En una PKI real se añadiría una CA intermedia para mantener la raíz fuera de línea; aquí se emite directamente desde la raíz, lo que el enunciado permite (la intermedia es opcional).

## 2. Configuración de un servidor HTTPS local

### 2.1. Servidor con Python (http.server)

Se usa *http.server* junto con el módulo *ssl*: se crea un contexto TLS de servidor, se cargan el certificado y la clave de *localhost* y se envuelve el socket. El servidor escucha en 127.0.0.1:8443 y sirve *www/index.html*.

**Figura 11**
*Script https_server.py*

terminal: ~/pki (https_server.py)

```
$ cat https_server.py
import http.server, ssl, functools
H = functools.partial(http.server.SimpleHTTPRequestHandler, directory="www")
httpd = http.server.HTTPServer(("127.0.0.1", 8443), H)
ctx = ssl.SSLContext(ssl.PROTOCOL_TLS_SERVER)
ctx.load_cert_chain("certs/localhost.cert.pem", "private/localhost.key.pem")
httpd.socket = ctx.wrap_socket(httpd.socket, server_side=True)
print("Sirviendo https://localhost:8443", flush=True)
httpd.serve_forever()
```

*Nota.* Contexto SSLContext con load_cert_chain y socket envuelto.

**Figura 12**
*Servidor HTTPS en ejecución*

terminal: ~/pki (servidor HTTPS)

```
$ python3 https_server.py
Sirviendo https://localhost:8443
127.0.0.1 - - [08/Nov/2026 01:07:29] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [08/Nov/2026 01:09:49] "GET / HTTP/1.1" 200 -
```

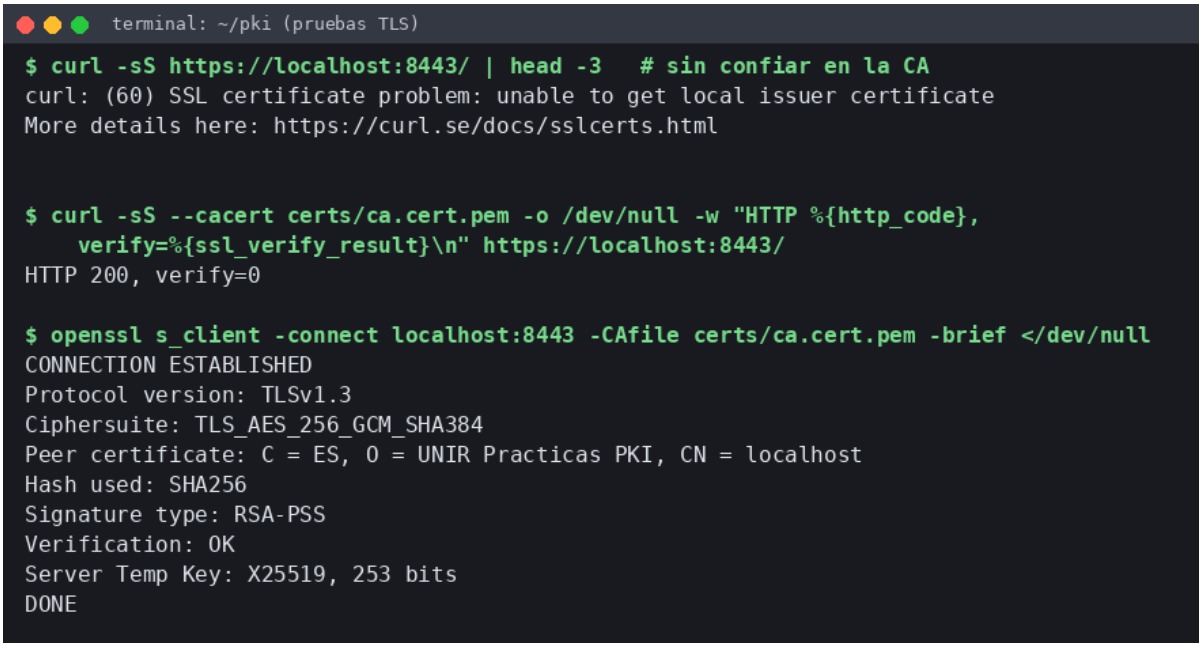
*Nota.* Registro de peticiones GET atendidas por el servidor.

### 2.2. Comprobación de la conexión TLS

Primero se prueba sin confiar en la CA: curl falla con el error 60 (*unable to get local issuer certificate*), prueba de que el cliente no reconoce al emisor. Al indicar la CA con *--cacert* la verificación es correcta (código 0) y *openssl s_client* confirma TLS 1.3, AES-256-GCM y *Verification: OK*.

**Figura 13**

*Pruebas con curl y openssl s_client*



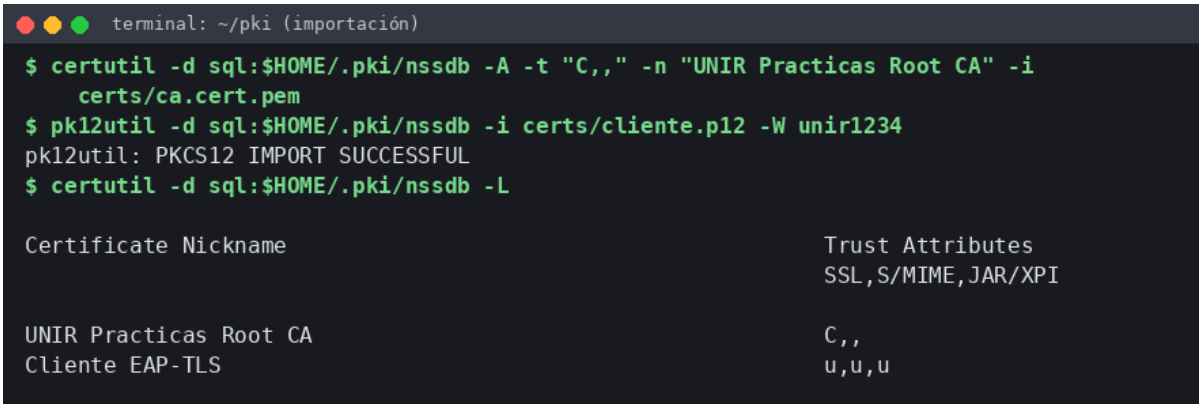

*Nota.* Sin CA falla; con la CA el certificado se verifica correctamente.

### 2.3. Importación de la CA en el navegador

Chromium en Linux usa la base NSS del usuario. Se importa la CA con *certutil -A -t "C,,"* (confianza para firmar certificados de servidor SSL) y el certificado de cliente con *pk12util*. En Firefox el equivalente gráfico es *Ajustes > Privacidad y seguridad > Ver certificados > Autoridades > Importar*, marcando «Confiar en esta CA para identificar sitios web».

**Figura 14**

*Importación de la CA y del certificado de cliente en la base NSS*

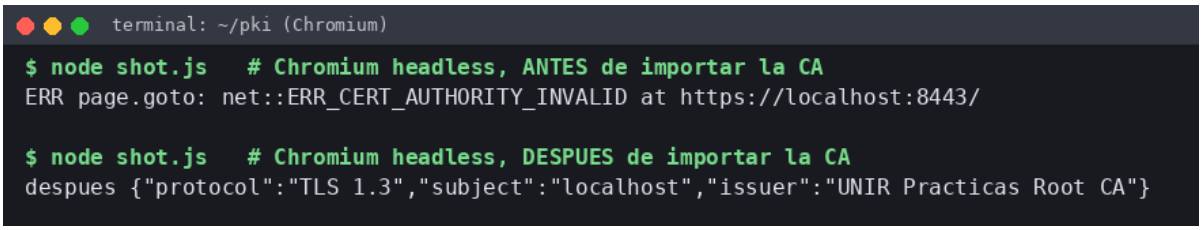



*Nota.* La CA queda con atributos C,, y el certificado de cliente con u,u,u (certificado personal con clave).

### 2.4. Acceso a https://localhost:8443 sin advertencias

Antes de la importación, Chromium bloquea el acceso con *net::ERR_CERT_AUTHORITY_INVALID*. Tras importar la CA, la página carga sin advertencias y el protocolo de seguridad informado por el navegador es TLS 1.3, con sujeto *localhost* y emisor *UNIR Practicas Root CA*.

**Figura 15**
*Resultado en Chromium antes y después de confiar en la CA*




*Nota.* Salida del script de automatización de Chromium (Playwright).

**Figura 16**
*https://localhost:8443 en Chromium tras importar la CA*




*Nota.* Captura de la página renderizada por Chromium en modo headless (sin barra de direcciones); la carga sin error y los detalles de seguridad de la Figura anterior demuestran la confianza.

## Conclusiones

Se ha construido una PKI funcional: CA raíz, certificado de servidor con SAN y certificado de cliente con clientAuth, verificados con *openssl verify*. La confianza depende exclusivamente de que la CA esté instalada en el almacén del cliente: sin ella el navegador rechaza la conexión y con ella la acepta sin advertencias. Para producción convendría una CA intermedia, revocación (CRL/OCSP) y custodiar la clave raíz fuera de línea.

## Referencias

Housley, R., Polk, W., Ford, W. y Solo, D. (2008). *Internet X.509 public key infrastructure certificate and certificate revocation list (CRL) profile* (RFC 5280). IETF. https://www.rfc-editor.org/rfc/rfc5280

OpenSSL Project. (2024). *openssl-ca: sample minimal CA application* (versión 3.0). https://docs.openssl.org/3.0/man1/openssl-ca/

OpenSSL Project. (2024). *openssl-req: PKCS#10 certificate request and certificate generating command* (versión 3.0). https://docs.openssl.org/3.0/man1/openssl-req/

Python Software Foundation. (s. f.). *ssl — TLS/SSL wrapper for socket objects*. Python 3 documentation. https://docs.python.org/3/library/ssl.html

Rescorla, E. (2018). *The Transport Layer Security (TLS) protocol version 1.3* (RFC 8446). IETF. https://www.rfc-editor.org/rfc/rfc8446

Simon, D., Aboba, B. y Hurst, R. (2008). *The EAP-TLS authentication protocol* (RFC 5216). IETF. https://www.rfc-editor.org/rfc/rfc5216